

Feuille de travaux dirigés n° 2

Synchronisation par verrous **— CORRECTION —**

Exercice 1 (Le dîner des philosophes ¹).

Des philosophes sont assis autour d'une table ronde. Devant chaque philosophe se trouve un bol de riz. Entre chaque paire de bols voisins se trouve une baguette. Ces philosophes occupent la majorité de leur temps à réfléchir, mais entre deux périodes d'intense réflexion, un philosophe a besoin de se nourrir. Pour manger, il doit commencer par prendre sa baguette de gauche et sa baguette de droite. Il peut alors manger un peu de riz. Une fois rassasié, il repose les deux baguettes. Faisons l'hypothèse que chaque philosophe mange pendant un temps fini et que le bol contient tellement de riz qu'il en restera toujours.

- 1a.** En utilisant un théorème du cours, proposez une solution deadlock-free.
- 1b.** Cette solution est-elle starvation-free ?

Exercice 2 (Les comptes en banque).

Une banque vous demande d'implémenter son nouveau système d'information. Chaque compte de ses clients est représenté par un numéro de compte entier et possède un certain solde entier en centimes d'euros. Au centre de son système réparti, les comptes de ses clients sont gérés par une classe `Banque` possédant les deux méthodes suivantes.

- La méthode `int solde(int compte)` retourne le solde du compte passé en paramètre.
- La méthode `boolean transfere(int montant, int de, int a)` vérifie que les fonds sont disponibles sur le compte `de` (sinon, elle s'arrête en retournant `false`), puis elle soustrait `montant` du compte `de` et l'ajoute au compte `a` de manière atomique, puis retourne `true`.

On supposera que l'ensemble des comptes est représenté par un tableau de taille fixe d'entiers positifs ou nuls. Lors de l'initialisation, on dotera chaque compte d'un montant fixe.

On souhaite une implémentation de `Banque` satisfaisant la contrainte suivante :

« Si deux transferts ne concernent pas les mêmes comptes, on ne veut pas empêcher le parallélisme entre les deux appels à `transfere`. »

- 2a.** Comment appelle-t-on la stratégie à mettre en place pour satisfaire la contrainte ?
- 2b.** Proposez une implémentation de `Banque` satisfaisant la contrainte.

1. Inspiration : E. W. Dijkstra, *Hierarchical ordering of sequential processes*. Acta informatica (1971)

Exercice 3 (Confection de tartes).

On veut modéliser le fonctionnement d'un atelier de pâtisserie. Pour préparer une tarte aux fraises, les pâtissiers doivent effectuer plusieurs étapes modélisées par des appels de fonctions comme ceci :

- Fond `preparerFond()` : prépare et étale un fond de tarte (20 minutes);
- Pate `cuire(Fond fond)` : cuit un fond de tarte (10 minutes);
- Creme `preparerCreme()` : prépare une crème pâtissière (10 minutes);
- Tarte `monter(Pate pate, Creme creme)` : étale la crème sur la pâte, et dispose les fruits (15 minutes).

3a. Proposez un programme Java parallélisant la création de tartes. Chaque thread représentera un pâtissier spécialisé sur une, et une seule, tâche. Vous embaucherez autant de pâtissier que nécessaire pour vous assurer qu'aucun d'entre eux ne reste inactif.

Exercice 4 (Arbre binaire de recherche).

On cherche à implémenter un arbre binaire de recherche (ABR) en se basant sur le code suivant. Identifier les problèmes de concurrence qui peuvent se poser, puis proposez un moyen de les résoudre.

```
public class ABR {
    private Node root = new Node();

    public boolean contains(int v) {
        return root.get(v).isSet;
    }

    public void insert(int v) {
        Node n = root.get(v);
        if(!n.isSet) n.setValue(v);
    }

    public void delete(int v) {
        Node n = root.get(v);
        if(!n.isSet) return;
        if(!n.smaller.isSet) n.copyNode(n.greater);
        else {
            Node predecessor = n.smaller;
            while(predecessor.greater.isSet)
                predecessor = predecessor.greater;
            n.value = predecessor.value;
            predecessor.copyNode(predecessor.smaller);
        }
    }
}

class Node {
    Node smaller = null;
    boolean isSet = false;
    int value;
    Node greater = null;

    void copyNode(Node n) {
        isSet = n.isSet; value = n.value; smaller = n.smaller; greater = n.greater;
    }

    void setValue(int v) {
        isSet = true; value = v; smaller = new Node(); greater = new Node();
    }

    Node get(int v) {
        if(!isSet || v == value) return this;
        if (v < value) return smaller.get(v);
        return greater.get(v);
    }
}
```